

Taller: Creación de APIs Simples para Usuarios

Unidad 2 – Tecnologías y Frameworks para el Desarrollo Móvil

Objetivo del Taller

El propósito de este taller es que los estudiantes diseñen, documenten y prueben **una API REST básica para la gestión de usuarios**, aplicando principios de arquitectura cliente-servidor, control de versiones y buenas prácticas de desarrollo backend.

1. Introducción

Las **APIs REST** son el puente de comunicación entre aplicaciones móviles y servidores. En este taller, aprenderás a:

- Definir endpoints básicos (GET , POST , PUT , DELETE).
- Implementar autenticación con **tokens JWT**.
- Validar y almacenar datos de usuarios.
- Probar los servicios con herramientas como **Postman** o **cURL**.

2. Requerimientos

Antes de iniciar, debes tener instalado en tu equipo:

- [Node.js](#) o [Python](#)
- Un editor de código (Visual Studio Code recomendado).
- Postman o extensión Thunder Client.
- Git para control de versiones.

3. Diseño de la API

Entidad: Usuario

Campo	Tipo	Descripción
id	UUID	Identificador único del usuario
nombre	String	Nombre completo del usuario
email	String	Correo electrónico (único)
password	String	Contraseña cifrada
fecha_creacion	DateTime	Fecha de registro

Endpoints

- POST /api/usuarios → Crear usuario.
- GET /api/usuarios → Listar usuarios.
- GET /api/usuarios/:id → Obtener usuario por ID.
- PUT /api/usuarios/:id → Actualizar datos de usuario.
- DELETE /api/usuarios/:id → Eliminar usuario.

4. Implementación

Ejemplo en Node.js (Express)

```
const express = require('express');
const app = express();
app.use(express.json());

let usuarios = [];

// Crear usuario
app.post('/api/usuarios', (req, res) => {
  const nuevo = { id: Date.now(), ...req.body };
  usuarios.push(nuevo);
  res.status(201).json(nuevo);
});

// Listar usuarios
app.get('/api/usuarios', (req, res) => {
  res.json(usuarios);
});

app.listen(3000, () => console.log("API en http://localhost:3000"));
```

Ejemplo en Python (FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel
import uuid

app = FastAPI()
usuarios = []

class Usuario(BaseModel):
    nombre: str
    email: str
    password: str

@app.post("/api/usuarios")
def crear_usuario(usuario: Usuario):
    nuevo = usuario.dict()
    nuevo["id"] = str(uuid.uuid4())
    usuarios.append(nuevo)
    return nuevo

@app.get("/api/usuarios")
def listar_usuarios():
    return usuarios
```

5. Autenticación y Seguridad

- Implementa **cifrado de contraseñas** con `bcrypt` (Node.js) o `passlib` (Python).
- Genera **tokens JWT** para sesiones seguras.
- Protege los endpoints con middleware o dependencias que validen el token.

6. Pruebas de la API

1. Abre Postman y crea una colección llamada `API Usuarios`.
2. Configura las solicitudes (`GET` , `POST` , etc.).
3. Prueba creando un usuario con el body JSON:

```
{  
  "nombre": "Juan Pérez",  
  "email": "juan@example.com",  
  "password": "123456"  
}
```

4. Verifica que los datos se guarden y puedas consultarlos.

7. Actividad Práctica

1. Implementa tu propia API de usuarios con al menos **3 endpoints funcionales**.
2. Documenta los endpoints en un archivo `README.md`.
3. Usa **GitHub/GitLab** para subir tu código y versionar tu proyecto.
4. Presenta evidencias de las pruebas en Postman (capturas de pantalla).

8. Preguntas de Reflexión

- ¿Qué ventajas ofrece separar frontend y backend?
- ¿Cómo mejora la seguridad el uso de JWT?
- ¿Qué problemas podrías tener si no documentas tu API?

Entrega: Sube tu proyecto a un repositorio y comparte el enlace con la documentación completa y capturas de prueba.